ACADEMIC RESEARCH CASE STUDY • LARGE-SCALE CODE COMPLEXITY

Brevity Is Not All You Need: A Large-Scale Empirical Study of Code Expansion, Defects, Complexity, and Stylometric Signatures in Human-Written vs. AI-Generated Code

Hassan Elkady — Computer Engineering Student, Arab Academy for Science, Technology and Maritime Transport (AAST)

August 2026 | Zenodo Dataset (DOI: 10.5281/zenodo.15423067): 190,593 Records / 762,372 Programs Evaluated

Abstract

Evaluating Large Language Model (LLM) code generation typically focuses on functional pass rates (e.g., HumanEval, MBPP) rather than non-functional dimensions such as code bloat, defect vulnerability, maintenance complexity, and stylometric visual formatting. This paper presents a large-scale empirical study evaluating **762,372 code snippets** across two primary benchmarks: (1) The Zenodo Large-Scale Dataset (10.5281/zenodo.15423067) comprising 190,593 problem records (86,748 Python and 103,845 Java) evaluating 762,372 programs produced by senior human developers and three frontier models (OpenAI ChatGPT, DeepSeek-Coder, Alibaba Qwen-Coder); and (2) The Frontier Task Benchmark (N=207) evaluating 147 zero-shot runs across 14 research tasks in Python and JavaScript produced by Google Gemini 3.5 Flash, OpenAI GPT-5.6 Sol, and Anthropic Claude Sonnet 4.6.

Empirical analysis resolves a key debate in LLM evaluation: **code bloat is prompt-scope dependent rather than an intrinsic defect of LLMs**. On single-function competitive coding prompts (Zenodo dataset), AI models generate concise code comparable to or smaller than human solutions (11.84 - 13.05 LOC Python vs. 14.58 LOC human; 11.02 - 14.84 LOC Java vs. 15.65 LOC human). Conversely, on multi-component task prompts (Frontier benchmark), AI models expand code length by **+221% to +264%** (48.13 ± 26.36 LOC vs. 15.00 ± 6.78 LOC human, Mann-Whitney U = 102.5, Holm-Bonferroni $p_{adj} = 3.32 \times 10^{-5}$, rank-biserial effect size $r_{rb} = +0.861$) due to **structural micro-fragmentation** (88.8% rate).

Crucially, complexity and maintainability analysis demonstrates: (1) Cyclomatic Complexity Reduction by **-22.6% to -42.6%** vs. Human code; (2) Control Flow Nesting Depth trimming; (3) Helper Subroutine Fragmentation in up to **26.93% of Java** (+630% vs Human) solutions; and (4) Halstead Maintainability Effort reduction by **-66.8% to -70.2%**.

Glossary of Technical & Statistical Terms for General Readers

- **Lines of Code (LOC)**: The total count of executable and structural code lines, excluding blank lines.
- **Cyclomatic Complexity (CC)**: A metric measuring the number of linearly independent paths through code control flow.
- **Halstead Maintainability Effort (E)**: A metric calculating mental effort required to understand code based on unique operators and operands.
- **Mann-Whitney U Test**: A non-parametric statistical test comparing two independent groups without assuming a bell-curve distribution.
- **Holm-Bonferroni FWER Correction (p_{adj})**: A procedure adjusting p-values to prevent false positive discoveries during multiple comparisons.
- **Rank-Biserial Correlation (r_{rb})**: An effect size metric (-1.0 to +1.0) indicating how strongly one group's values exceed another.
- **Structural Micro-Fragmentation**: The tendency of synthetic models to decompose simple logic into multiple helper sub-functions or auxiliary class wrappers.

2. Quantitative Results & Statistical Significance

2.1 Zenodo Complexity & Maintainability Analysis (762,372 Programs)

Language	Author / Model	Cyclomatic Complexity (CC)	Mean Nesting Depth	Helper Function Rate (%)	Halstead Effort (E)
Python	**Human Developer**	**3.31 ± 4.12** [P95: 10.0]	**1.37 ± 0.88** [17.7% high nest]	**3.85%**	**19,103 ± 42,100**
Python	**OpenAI ChatGPT**	2.56 ± 2.89 (-22.6%)	1.02 ± 0.65	12.14%	8,421 ± 18,200 (-55.9%)
Python	**DeepSeek-Coder**	**2.14 ± 2.15** (-35.3%)	**0.90 ± 0.58** [7.3% high nest]	**20.59%** (+435%)	**5,699 ± 12,400** (-70.2%)
Python	**Alibaba Qwen-Coder**	2.56 ± 3.01 (-22.5%)	1.05 ± 0.70	9.82%	7,942 ± 17,100 (-58.4%)
Java	**Human Developer**	**3.94 ± 5.01** [P95: 11.0]	**1.52 ± 0.95**	**3.69%**	**28,320 ± 61,200**
Java	**OpenAI ChatGPT**	2.79 ± 3.12 (-29.2%)	1.15 ± 0.72	16.42%	12,410 ± 26,500 (-56.2%)
Java	**DeepSeek-Coder**	**2.26 ± 2.45** (-42.6%)	**0.98 ± 0.61**	**26.93%** (+630%)	**9,392 ± 19,800** (-66.8%)
Java	**Alibaba Qwen-Coder**	2.43 ± 2.88 (-38.4%)	1.04 ± 0.68	21.15%	7,824 ± 16,900 (-72.4%)

2.2 Frontier Model Task Study: Human Reference (n=10) vs. Frontier LLMs (N=147)

Stylometric Metric	Human Reference (n=10)	Frontier LLMs (N=147)	Mann-Whitney U	Raw p-val	Holm-Bonferroni p_{adj}	Rank-Biserial Effect Size (r_{rb})	FWER Significance
Lines of Code (LOC)	15.00 ± 6.78 [11.1, 18.9]	48.13 ± 26.36 [43.8, 52.5]	102.5	p = 5.53e-06	**p_{adj} = 3.32e-05**	**r_{rb} = +0.861 (Massive)**	**Significant (p < 0.01)**
Comment Density (%)	1.43% ± 4.52% [0.0, 4.3]	9.56% ± 10.82% [7.8, 11.3]	280.0	p = 3.12e-04	**p_{adj} = 0.0018**	**r_{rb} = +0.621 (Large)**	**Significant (p < 0.01)**
Explicit Type Annotations	1.50 ± 1.35 [0.7, 2.3]	8.12 ± 8.95 [6.6, 9.6]	212.0	p = 6.15e-05	**p_{adj} = 0.0003**	**r_{rb} = +0.712 (Large)**	**Significant (p < 0.01)**
Vertical Whitespace (%)	5.54% ± 6.95% [1.8, 9.8]	15.92% ± 4.88% [15.1, 16.7]	172.0	p = 3.10e-05	**p_{adj} = 0.0002**	**r_{rb} = +0.765 (Massive)**	**Significant (p < 0.01)**


Figure 1: Mean Lines of Code (LOC) Expansion Across Author Groups

3. Key Scientific Synthesis & Discussion

1. Micro-Fragmentation & Maintainability: AI models fragment logic into helper subroutines in 20.59% (Python) and 26.93% (Java) of solutions (vs. ~3.7% Human), reducing Cyclomatic Complexity (-22.6% to -42.6%) and Halstead Maintainability Effort (-66.8% to -70.2%).
2. Security vs Maintainability Trade-Off: Despite lower cyclomatic complexity, static security analysis reveals that ChatGPT and DeepSeek-Coder introduce 5x to 7x more command injection flaws (shell=True) and hardcoded credentials (0.46% vs 0.02% Human).

4. Conclusion & References

Evaluating 762,372 code snippets demonstrates that LLM code bloat is prompt-scope dependent while AI models reduce cyclomatic complexity and Halstead effort through structural micro-fragmentation.

References: Zenodo Dataset (2025) DOI: 10.5281/zenodo.15423067; Binkley et al. (2023) TSE; Jesse et al. (2023) ISSTA; Kabir et al. (2023) EMSE; Nguyen et al. (2023) ICSE; Ugare et al. (2024) PACMPL.